

# Autonomous Drone Simulation for Mine Detection: Technical Report

Remote Lead Team

December 15, 2025

## Abstract

This report details the development and implementation of an autonomous control system for a quadcopter tasked with detecting simulated landmines using thermal imaging. The project utilizes a Software-In-The-Loop (SITL) environment to validate navigation, computer vision, and decision-making logic without physical hardware risks. Key achievements include the implementation of a comprehensive "Lawnmower" search pattern, a custom thermal sensor simulation, and a robust Finite State Machine (FSM) for target verification.

## Contents

|          |                                                                          |          |
|----------|--------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Simulation Environment Setup</b>                                      | <b>2</b> |
| 1.1      | Software Stack . . . . .                                                 | 2        |
| 1.2      | Execution Workflow . . . . .                                             | 2        |
| <b>2</b> | <b>Core Logic &amp; Algorithm</b>                                        | <b>3</b> |
| 2.1      | Navigation: The "Lawnmower" Search Pattern . . . . .                     | 3        |
| 2.2      | Sensor Simulation ( <code>field_simulation.py</code> ) . . . . .         | 3        |
| 2.3      | Detection & Verification Algorithm ( <code>mission.py</code> ) . . . . . | 3        |
| <b>3</b> | <b>Key Technical Challenges Solved</b>                                   | <b>4</b> |
| 3.1      | Coordinate System Unification . . . . .                                  | 4        |
| 3.2      | The "Stuck Drone" Issue . . . . .                                        | 4        |
| 3.3      | False Positive Rejection . . . . .                                       | 4        |
| <b>4</b> | <b>Visual Verification</b>                                               | <b>5</b> |

# 1 Simulation Environment Setup

The development environment leverages the ArduPilot ecosystem to create a high-fidelity simulation of drone physics and communication protocols.

## 1.1 Software Stack

- **ArduPilot SITL (Software In The Loop):** The core simulation engine that runs the actual ArduCopter flight firmware on a standard PC, simulating physics, gravity, and inertial sensors.
- **MAVProxy:** A command-line Ground Control Station (GCS) that functions as a MAVLink router. It splits the single telemetry stream from the drone into multiple outputs, allowing simultaneous connection of the control script, dashboard, and visualizer.
- **QGroundControl (QGC):** The primary visual interface used for monitoring the drone's position on a satellite map, configuring parameters, and observing flight status.
- **Python (DroneKit):** The "Companion Computer" logic. A custom Python script uses the DroneKit library to communicate with the flight controller via MAVLink, executing high-level autonomous commands.

## 1.2 Execution Workflow

The simulation is initialized using three distinct terminal processes:

### 1. Drone Physics (SITL):

```
1 ./arducopter -S -I0 --home 22.99982,72.00,0,180 --model "+" --speedup 1
2
```

This command starts the virtual drone at a specific latitude in Gujarat, India. The latitude is shifted 20 meters South to align the start of the simulated field with the local coordinate system origin.

### 2. Telemetry Routing (MAVProxy):

```
1 mavproxy.py --master tcp:127.0.0.1:5760 --out udp:127.0.0.1:14550 --out udp
2 :127.0.0.1:14551 --out udp:127.0.0.1:14552
```

This routes the drone's data to:

- Port 14550: QGroundControl
- Port 14551: Mission Script (mission.py)
- Port 14552: Live Dashboard (dashboard.py)

### 3. Ground Station (QGC):

```
1 ./QGroundControl.AppImage
2
```

Provides the visual map interface.

## 2 Core Logic & Algorithm

### 2.1 Navigation: The "Lawnmower" Search Pattern

To ensure complete coverage of the 40m x 40m field, the drone executes a systematic sweep pattern:

1. **Initialization:** The drone takes off and navigates to the South-East corner of the field (Local coordinates: East=20m, North=0m).
2. **Sweep:** It flies a straight path North (+40m).
3. **Shift:** Upon reaching the northern edge, it shifts laterally West (Left) by 2 meters (defined as SWEEP\_STEP).
4. **Reverse:** It flies back South to the starting baseline.
5. **Repeat:** This loop continues until the drone's position reaches the Western edge of the field (East=-20m).

### 2.2 Sensor Simulation (`field_simulation.py`)

A custom Python module was developed to simulate a thermal camera, as hardware-in-the-loop thermal data was unavailable.

- **Thermal Map Generation:** Creates a 40m x 40m grid representing ground temperatures.
- **Noise Injection:** Adds random Gaussian noise to a base temperature of 22°C to simulate realistic soil texture variations.
- **Target Modeling:**
  - **Mines:** Modeled as perfect circular heat blobs with a peak temperature of  $\approx 30^{\circ}\text{C}$ .
  - **Rocks (Clutter):** Modeled as irregular, rotated elliptical blobs with a lower peak temperature ( $\approx 25\text{--}27^{\circ}\text{C}$ ) to test false-positive rejection.
- **Optical Simulation:** The camera view is dynamically generated based on altitude:
  - **High Altitude (2m):** Uses pixel binning to lower resolution, simulating a wide Field of View (FOV) where distinct shapes are blurred.
  - **Low Altitude (0.5m):** Provides high-resolution raw data for accurate shape verification.

### 2.3 Detection & Verification Algorithm (`mission.py`)

The control logic is implemented as a Finite State Machine (FSM):

1. **SEARCH:** The drone traverses the lawnmower path at an altitude of 2 meters.
2. **DETECT:** The simulated thermal camera polls for pixel values exceeding a threshold of **29°C**. If triggered:
  - The drone switches to **GUIDED** mode to hold position.
  - **Visual Servoing:** The script calculates the pixel offset of the target from the center of the image and commands the drone to move until it is directly hovering over the target.

3. **DESCEND:** The drone drops to an altitude of 0.5 meters to obtain a high-resolution thermal image.
4. **VERIFY:**
  - A high-resolution image is captured.
  - **Computer Vision:** The OpenCV library is used to apply a Hough Circle Transform.
  - **Classification:** If a circle is detected, the target is logged as a **Confirmed Mine**. If the shape is irregular, it is dismissed as a **False Alarm**.
5. **RECOVER:**
  - The drone ascends back to 2 meters.
  - **"Nudge" Maneuver:** The drone flies blindly forward by 3 meters. This critical step forces the target out of the camera's field of view, preventing the system from re-detecting the same mine in an infinite loop.
  - The search mode resumes.

### 3 Key Technical Challenges Solved

#### 3.1 Coordinate System Unification

**Problem:** The drone operates in geodetic coordinates (Latitude/Longitude), while the thermal simulation and path planning operate in a local Cartesian grid (Meters X/Y). **Solution:** A custom `get_relative_meters` function was implemented using the Haversine formula. This converts real-time GPS telemetry into local North/East offsets relative to the home position, ensuring perfect alignment between the drone's physical location and the simulated thermal map.

#### 3.2 The "Stuck Drone" Issue

**Problem:** The drone would occasionally hang indefinitely while attempting to center itself over a target. **Cause:** The original movement logic waited for the distance to the target to be less than 0.3 meters. Due to simulated wind and GPS drift physics, the drone often could not achieve this tight tolerance. **Solution:** A **Timeout** mechanism (15 seconds) was added to the movement loop, and the distance tolerance was relaxed to  $< 1.0$  meter. This ensures the mission proceeds even if perfect position holding is not possible.

#### 3.3 False Positive Rejection

**Problem:** Initial tests showed the drone misidentifying heated rocks as mines. **Solution:** A robust two-stage filter was implemented:

1. **Thermal Threshold:** The detection threshold was raised to 29°C, effectively filtering out cooler rocks.
2. **Geometric Verification:** The Hough Circle Transform serves as the final arbiter, rejecting any high-heat targets that do not match the geometric profile of a mine.

## 4 Visual Verification

To independently verify the autonomous behavior, a real-time dashboard (`dashboard.py`) was built using `matplotlib`.

- It connects to a parallel telemetry stream from MAVProxy.
- It renders the "Ground Truth" thermal map generated by the simulation seed.
- It plots the drone's live position and path as an overlay.

This tool proved invaluable for visually confirming that the drone was executing the lawnmower pattern correctly and stopping precisely over the generated mine locations.